MTM4692

Spring 2026

Fettah KIRAN
Department of AI and Data Engineering
fkiran@yildiz.edu.tr

SQL Programming

Variables · Control Structures · Error Handling — MySQL & SQLite

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:15	Week 9 recap poll	Review
1:15 – 1:50	Variables & Assignments (MySQL)	**Theory** 📖
1:50 – 2:15	Control Structures: IF, CASE, Loops	**Theory** 📖
2:15 – 2:30	☕ Break (15 min)	
2:30 – 3:05	Error Handling: DECLARE HANDLER & TRY...CATCH	**Theory** 📖
3:05 – 3:20	DELIMITER & Compound Statements	**Theory** 📖
3:20 – 3:40	Hands-on: SQLite Equivalents	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Week 9 Quick Recap

Concept	Key Point
Pivot	Rows → Columns using CASE WHEN + GROUP BY
Unpivot	Columns → Rows using UNION ALL
Conditional aggregation	`MAX(CASE WHEN col='X' THEN val END)`
Numeric pivot	`SUM(CASE WHEN ... THEN amount ELSE 0 END)`
Count cross-tab	`COUNT(CASE WHEN ... THEN 1 END)`
Dynamic pivot	Not native in SQLite; requires scripting

📖 PART 1 — THEORY

Variables & Assignments

MySQL vs SQLite: Programming Support

Feature	MySQL	SQLite
Variables	`DECLARE`, `SET`, `@var`	No native variables
IF/ELSE	Full support in procedures	CASE expression only
Loops	WHILE, REPEAT, LOOP	Recursive CTEs
Stored procedures	Full support	Not supported
Functions	Full support	Not supported natively
Cursors	Full support	Not supported
DELIMITER	Required for procedures	Not applicable
Error handling	DECLARE HANDLER, SIGNAL	Limited

MySQL Variables: Local Variables

```sql
-- DECLARE: local variables inside a stored program with a data type
DELIMITER //
CREATE PROCEDURE calc_bonus()
BEGIN
    DECLARE base_salary DECIMAL(10,2);
    DECLARE bonus_rate DECIMAL(5,2);
    DECLARE total_bonus DECIMAL(10,2);

    SET base_salary = 50000.00;
    SET bonus_rate = 0.10;
    SET total_bonus = base_salary * bonus_rate;

    SELECT total_bonus AS bonus;
END //
DELIMITER ;
```

> `DECLARE` must come **before** any other statements in BEGIN...END

MySQL Variables: User-Defined Variables

```sql
-- @variable: user-defined (session-scoped)
SET @student_count = (SELECT COUNT(*) FROM student);
SET @avg_gpa = (SELECT AVG(gpa) FROM student);

SELECT @student_count AS total_students,
       @avg_gpa AS average_gpa;

-- Use in queries
SELECT first_name, gpa,
       gpa - @avg_gpa AS diff_from_avg
FROM student
WHERE gpa > @avg_gpa;
```

Local vs User-Defined:

Feature	Local (`DECLARE`)	User-Defined (`@var`)
Scope	BEGIN...END block	Entire session
Needs DECLARE?	Yes	No
Data type?	Specified	Auto-detected
Used in procedures?	Yes	Yes
Used in ad-hoc SQL?	No	Yes

MySQL: SELECT INTO

```sql
--  primarily used to create a new table and populate it with data from an existing table in a single step
DELIMITER //
CREATE PROCEDURE get_student_info(IN sid INT)
BEGIN
    DECLARE sname VARCHAR(100);
    DECLARE sgpa DECIMAL(3,2);

    SELECT CONCAT(first_name, ' ', last_name), gpa
    INTO sname, sgpa
    FROM student
    WHERE student_id = sid;

    SELECT sname AS name, sgpa AS gpa;
END //
DELIMITER ;

CALL get_student_info(1);


-- `SELECT ... INTO` assigns column values directly to variables.
```

📖 PART 2 — THEORY

Control Structures

IF / ELSEIF / ELSE (MySQL)

```sql
DELIMITER //
CREATE PROCEDURE grade_student(IN student_gpa DECIMAL(3,2))
BEGIN
    DECLARE grade_letter VARCHAR(2);

    IF student_gpa >= 3.7 THEN
        SET grade_letter = 'A';
    ELSEIF student_gpa >= 3.3 THEN
        SET grade_letter = 'B+';
    ELSEIF student_gpa >= 3.0 THEN
        SET grade_letter = 'B';
    ELSEIF student_gpa >= 2.0 THEN
        SET grade_letter = 'C';
    ELSE
        SET grade_letter = 'F';
    END IF;

    SELECT student_gpa AS gpa, grade_letter AS grade;
END //
DELIMITER ;

CALL grade_student(3.5);  -- Returns: B+
```

CASE Statement (MySQL — procedural)

```sql
-- CASE in a stored procedure (statement form, not expression)
DELIMITER //
CREATE PROCEDURE categorize_dept(IN dept VARCHAR(50))
BEGIN
    DECLARE category VARCHAR(50);

    CASE dept
        WHEN 'Computer Science' THEN SET category = 'Engineering';
        WHEN 'Mathematics' THEN SET category = 'Science';
        WHEN 'Physics' THEN SET category = 'Science';
        WHEN 'English' THEN SET category = 'Humanities';
        ELSE SET category = 'Other';
    END CASE;

    SELECT dept, category;
END //
DELIMITER ;
```

🧠 Exam Focus: IF vs CASE

Q: In MySQL, where can `IF ... ELSEIF ... END IF` be used?

A) Anywhere in SQL

B) Only in SELECT statements

C) **Only inside stored procedures, functions, or triggers** ✅

D) Only in WHERE clauses

Q: What is the difference between CASE statement and CASE expression?

A) They are identical

B) **CASE statement is used in procedures with SET; CASE expression is used in SELECT** ✅

C) CASE statement is faster

D) CASE expression only works with numbers

WHILE Loop (MySQL)

Syntax:

```
WHILE condition DO
    -- statements
END WHILE;
```

WHILE Loop (MySQL)

```sql
DELIMITER //
CREATE PROCEDURE count_to_ten()
BEGIN
    DECLARE i INT DEFAULT 1;
    DECLARE result TEXT DEFAULT '';

    WHILE i <= 10 DO
        SET result = CONCAT(result, i, ' ');
        SET i = i + 1;
    END WHILE;

    SELECT result;
END //
DELIMITER ;

CALL count_to_ten();
-- Result: "1 2 3 4 5 6 7 8 9 10 "
```

REPEAT...UNTIL Loop (MySQL)

```sql
DELIMITER //
CREATE PROCEDURE factorial(IN n INT)
BEGIN
    DECLARE result BIGINT DEFAULT 1;
    DECLARE counter INT DEFAULT 1;

    REPEAT
        SET result = result * counter;
        SET counter = counter + 1;
    UNTIL counter > n
    END REPEAT;

    SELECT n AS input, result AS factorial;
END //
DELIMITER ;

CALL factorial(5);  -- Result: 120
```

Key difference from WHILE:

- **WHILE**: Checks condition **before** each iteration (may run 0 times)
- **REPEAT**: Checks condition **after** each iteration (runs at least once)

LOOP with LEAVE (MySQL)

```
DELIMITER //
CREATE PROCEDURE find_first_honor(OUT found_name VARCHAR(100))
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE sname VARCHAR(100);
    DECLARE sgpa DECIMAL(3,2);
    DECLARE cur CURSOR FOR SELECT first_name, gpa FROM student ORDER BY gpa DESC;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
    OPEN cur;
    search_loop: LOOP
        FETCH cur INTO sname, sgpa;
        IF done THEN
            LEAVE search_loop;
        END IF;
        IF sgpa >= 3.5 THEN
            SET found_name = sname;
            LEAVE search_loop;
        END IF;
    END LOOP;

    CLOSE cur;
END //
DELIMITER ;

> `LEAVE` exits a named loop. `ITERATE` skips to the next iteration (like `continue`).
```

🧠 Exam Focus: Loops

Q: What is the difference between WHILE and REPEAT loops in MySQL?

A) WHILE is faster than REPEAT

B) **WHILE checks the condition before each iteration; REPEAT checks after** ✅

C) REPEAT can only run once

D) WHILE cannot use variables

Q: What does `LEAVE loop_name` do?

A) Pauses the loop

B) Restarts the loop

C) **Exits the named loop immediately** ✅

D) Skips to the next iteration

☕ BREAK (10 minutes)

Error Handling

Why Error Handling?

Real-world scenarios without error handling:

```sql
-- ❌ BAD: No error handling
INSERT INTO student (first_name, email) VALUES ('John', 'john@ytu.edu.tr');
-- If email already exists → CRASH! Database inconsistent.

UPDATE enrollment SET grade = 'A' WHERE student_id = 999;
-- If student doesn't exist → Silent fail. Did it work?

DECLARE @total DECIMAL(10,2);
SELECT @total = salary / years_employed FROM employees;
-- If years_employed = 0 → Divide by zero error. Procedure stops.
```

With error handling (best practice):

```
✅ Catch exceptions before they crash the application
✅ Provide meaningful error messages to users
✅ Rollback partial transactions (atomicity)
✅ Log errors for debugging
✅ Gracefully degrade or retry operations
✅ Maintain data integrity
```

Problem	Without Error Handling	With Error Handling
Duplicate insert	Application crashes	Caught & user notified
Missing record	Silent failure	Caught & logged
Division by zero	Stops execution	Caught & handled
Constraint violation	Partial update	Rollback everything
Network timeout	No retry	Can retry logic

What is DECLARE HANDLER?

Definition:

A **DECLARE HANDLER** statement defines a **named error handler** that automatically executes when a specific error condition occurs inside a stored procedure, function, or trigger.

Syntax:

```
DECLARE [action] HANDLER FOR [condition]
    statement;
```

Part	Meaning	Example
action	What to do when error occurs	`CONTINUE` or `EXIT`
condition	Which error(s) to catch	`1062` (error code) or `NOT FOUND`
statement	What to execute	`SET flag = 1;` or any SQL statement

MySQL Error Codes (most common)

Code	Error	Cause	Handler
1062	Duplicate entry	UNIQUE constraint violated	`DECLARE HANDLER FOR 1062`
1048	Column cannot be null	NOT NULL constraint violated	`DECLARE HANDLER FOR 1048`
1452	FK constraint fails	Parent record doesn't exist	`DECLARE HANDLER FOR 1452`
1406	Data too long	VARCHAR/CHAR overflow	`DECLARE HANDLER FOR 1406`
1216	FK constraint fails (child)	Child record exists, can't delete parent	`DECLARE HANDLER FOR 1216`
1205	Lock wait timeout	Too many concurrent locks	`DECLARE HANDLER FOR 1205`

SQL Server Error Numbers (common)

Code	Error	Cause	Recovery
2627	Unique constraint violation	Duplicate key	Handle in CATCH, retry
547	FK constraint violation	Missing parent/child	Validate before insert/delete
8134	Divide by zero	denominator = 0	Check before division
208	Invalid object name	Table/column doesn't exist	Verify schema
229	Permission denied	User lacks rights	Check role/permissions
50000+	User-defined error	Custom THROW statement	Business logic error

DECLARE HANDLER (MySQL)

```
DELIMITER //
CREATE PROCEDURE safe_insert(IN sname VARCHAR(100), IN semail VARCHAR(100))
BEGIN
    DECLARE duplicate_error INT DEFAULT 0;

    -- Handler for duplicate key error (MySQL error 1062)
    DECLARE CONTINUE HANDLER FOR 1062
        SET duplicate_error = 1;

    INSERT INTO student (first_name, email) VALUES (sname, semail);

    IF duplicate_error THEN
        SELECT 'Error: Email already exists' AS message;
    ELSE
        SELECT 'Student inserted successfully' AS message;
    END IF;
END //
DELIMITER ;
```

Handler types:

- `CONTINUE`: Execute handler and continue the procedure
- `EXIT`: Execute handler and exit the BEGIN...END block

SQL Server: TRY...CATCH

```sql
-- TRY...CATCH structure (SQL Server equivalent)
CREATE PROCEDURE validate_gpa_sqlserver
    @gpa DECIMAL(3,2)
AS
BEGIN
    BEGIN TRY
        -- Validation logic
        IF @gpa < 0 OR @gpa > 4.0
        BEGIN
            THROW 50001, 'GPA must be between 0 and 4.0', 1;
        END;

        SELECT @gpa AS valid_gpa;
    END TRY
    BEGIN CATCH
        -- Error handling block
        SELECT
            ERROR_NUMBER() AS error_code,
            ERROR_MESSAGE() AS error_message,
            ERROR_SEVERITY() AS severity,
            ERROR_STATE() AS error_state;
    END CATCH;
END;

EXEC validate_gpa_sqlserver @gpa = 5.0;
-- Output: error_code: 50001, error_message: GPA must be between 0 and 4.0
```

TRY...CATCH with Transaction Handling (SQL Server)

```sql
-- TRY...CATCH with explicit error logging and rollback
CREATE PROCEDURE safe_insert_sqlserver
    @sname VARCHAR(100),
    @semail VARCHAR(100)
AS
BEGIN
    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO student (first_name, email)
        VALUES (@sname, @semail);

        COMMIT TRANSACTION;
        SELECT 'Student inserted successfully' AS message;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;

        DECLARE @ErrorMsg NVARCHAR(MAX) = ERROR_MESSAGE();
        DECLARE @ErrorNum INT = ERROR_NUMBER();

        -- Handle specific errors
        IF @ErrorNum = 2627  -- Unique constraint violation
        BEGIN
            SELECT 'Error: Email already exists' AS message;
        END
        ELSE
        BEGIN
            SELECT 'Error: ' + @ErrorMsg AS message;
            THROW;  -- Re-throw the error
        END;
    END CATCH;
END;

EXEC safe_insert_sqlserver @sname = 'John', @semail = 'john@ytu.edu.tr';
```

SQL Server: Error Functions in CATCH Block

```sql
-- Error functions available only within CATCH block
BEGIN TRY
    SELECT 1/0;  -- Division by zero
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS err_num,           -- 8134
        ERROR_SEVERITY() AS err_severity,    -- 16 (0-25)
        ERROR_STATE() AS err_state,          -- 1
        ERROR_PROCEDURE() AS err_proc,       -- NULL (not in a procedure)
        ERROR_LINE() AS err_line,            -- Line number where error occurred
        ERROR_MESSAGE() AS err_msg;          -- "Divide by zero error encountered."
END CATCH;
```

Error Functions Reference:

Function	Returns	Example
`ERROR_NUMBER()`	Error code (int)	8134 (divide by zero)
`ERROR_SEVERITY()`	Severity 0-25 (int)	16 (user error), 20+ (severe)
`ERROR_STATE()`	Error sub-state (int)	1, 2, 3...
`ERROR_PROCEDURE()`	Procedure name (nvarchar)	'usp_InsertData' or NULL
`ERROR_LINE()`	Line number (int)	5, 12, 42...
`ERROR_MESSAGE()`	Error text (nvarchar)	'Divide by zero error encountered.'

> 💡 These functions are **only available inside CATCH blocks** and return NULL outside.

🧠 Exam Focus: Error Handling

Q: What does `DECLARE CONTINUE HANDLER FOR 1062` do in MySQL?

A) Stops the procedure when error 1062 occurs

B) **Handles error 1062 and continues executing the procedure** ✅

C) Prevents error 1062 from ever occurring

D) Logs error 1062 to a file

Q: What is the main difference between MySQL DECLARE HANDLER and SQL Server TRY...CATCH?

A) They do the same thing identically

B) **MySQL is event-specific handlers; SQL Server catches block of statements** ✅

C) TRY...CATCH only works with SELECT statements

D) DECLARE HANDLER is more powerful

> **Q:** In SQL Server, what does `ERROR_NUMBER()` return inside a CATCH block?
>
> A) The line number of the error
>
> B) **The SQL error code/number** ✅
>
> C) The severity level
>
> D) The error message text

DELIMITER & Compound Statements

Why Do We Need DELIMITER?

```
-- Problem: MySQL client treats ; as end of statement
-- But procedures CONTAIN semicolons!

CREATE PROCEDURE hello()
BEGIN
    SELECT 'Hello';  -- ← MySQL thinks the procedure ends here!
END;                 -- ← Never reached

-- Solution: Change the delimiter temporarily
DELIMITER //

CREATE PROCEDURE hello()
BEGIN
    SELECT 'Hello';  -- ; no longer ends the statement
END //               -- // ends the CREATE PROCEDURE

DELIMITER ;          -- Restore normal delimiter
```

> 💡 `DELIMITER //` tells MySQL: "Don't treat `;` as the end of my statement until I say `//`."

🧠 Exam Focus: DELIMITER

> **Q:** Why is `DELIMITER //` needed before creating a stored procedure in MySQL?
>
> A) It makes the procedure run faster
>
> B) **It prevents the MySQL client from interpreting `;` inside the procedure as the end of the CREATE statement** ✅
>
> C) It defines the procedure's return type
>
> D) It is required by SQL standard

SQLite Equivalents

SQLite: No Variables, But We Have CTEs!

```sql
-- MySQL: SET @avg_gpa = (SELECT AVG(gpa) FROM student);
-- SQLite equivalent using CTE:

WITH vars AS (
    SELECT AVG(gpa) AS avg_gpa,
           COUNT(*) AS total_students
    FROM student
)
SELECT s.first_name, s.gpa,
       v.avg_gpa,
       ROUND(s.gpa - v.avg_gpa, 2) AS diff
FROM student s, vars v
WHERE s.gpa > v.avg_gpa
ORDER BY s.gpa DESC;
```

> 💡 In SQLite, use CTEs as "variable containers".

SQLite: CASE Expression Instead of IF/ELSE

```sql
-- MySQL IF/ELSE → SQLite CASE
SELECT first_name, gpa,
    CASE
        WHEN gpa >= 3.7 THEN 'A'
        WHEN gpa >= 3.3 THEN 'B+'
        WHEN gpa >= 3.0 THEN 'B'
        WHEN gpa >= 2.0 THEN 'C'
        ELSE 'F'
    END AS letter_grade,
    CASE
        WHEN gpa >= 3.5 THEN 'Honor Roll'
        WHEN gpa >= 2.0 THEN 'Good Standing'
        ELSE 'Academic Probation'
    END AS status
FROM student
ORDER BY gpa DESC;
```

SQLite: Recursive CTE Instead of Loops

```sql
-- MySQL WHILE loop → SQLite recursive CTE

-- Factorial using recursive CTE
WITH RECURSIVE factorial AS (
    SELECT 1 AS n, 1 AS fact
    UNION ALL
    SELECT n + 1, fact * (n + 1)
    FROM factorial WHERE n < 10
)
SELECT n, fact AS factorial FROM factorial;

-- Cumulative sum (simulates loop accumulation)
WITH RECURSIVE running AS (
    SELECT student_id, first_name, gpa,
           ROW_NUMBER() OVER (ORDER BY student_id) AS rn
    FROM student
)
SELECT first_name, gpa,
       SUM(gpa) OVER (ORDER BY rn) AS running_gpa_sum,
       ROUND(AVG(gpa) OVER (ORDER BY rn), 2) AS running_avg
FROM running;
```

SQLite: Error Handling with Constraints

```sql
-- SQLite's "error handling" is via constraints + OR clauses

-- ON CONFLICT strategies
CREATE TABLE IF NOT EXISTS safe_table (
    id INTEGER PRIMARY KEY,
    email TEXT UNIQUE,
    name TEXT NOT NULL
);

-- INSERT OR IGNORE: skip on conflict
INSERT OR IGNORE INTO safe_table (email, name)
VALUES ('test@test.com', 'Test');
INSERT OR IGNORE INTO safe_table (email, name)
VALUES ('test@test.com', 'Duplicate');  -- Silently ignored

-- INSERT OR REPLACE: replace on conflict
INSERT OR REPLACE INTO safe_table (id, email, name)
VALUES (1, 'test@test.com', 'Updated');

-- INSERT OR ABORT (default): stop and rollback
-- INSERT INTO safe_table VALUES (1, 'test@test.com', 'Fail');

SELECT * FROM safe_table;
DROP TABLE safe_table;
```

💻 Practice: Building Reports with CTEs

```sql
-- Comprehensive student report using CTEs (no variables needed)
WITH
    stats AS (
        SELECT AVG(gpa) AS avg_gpa,
               MIN(gpa) AS min_gpa,
               MAX(gpa) AS max_gpa,
               COUNT(*) AS total
        FROM student
    ),
    ranked AS (
        SELECT first_name, last_name, gpa,
               DENSE_RANK() OVER (ORDER BY gpa DESC) AS overall_rank,
               NTILE(4) OVER (ORDER BY gpa DESC) AS quartile
        FROM student
    )
SELECT r.first_name, r.last_name, r.gpa,
       r.overall_rank,
       CASE r.quartile
           WHEN 1 THEN 'Top 25%'
           WHEN 2 THEN 'Upper Middle'
           WHEN 3 THEN 'Lower Middle'
           WHEN 4 THEN 'Bottom 25%'
       END AS bracket,
       ROUND(r.gpa - s.avg_gpa, 2) AS vs_average,
       ROUND(100.0 * (r.gpa - s.min_gpa) / (s.max_gpa - s.min_gpa), 1)
           AS percentile
FROM ranked r, stats s
ORDER BY r.overall_rank;
```

💻 Practice: Conditional Logic in Queries

```sql
-- Complex CASE with multiple conditions
SELECT first_name, gpa, dept_id,
    CASE
        WHEN gpa >= 3.5 AND dept_id = 1 THEN 'CS Honor'
        WHEN gpa >= 3.5 AND dept_id = 2 THEN 'Math Honor'
        WHEN gpa >= 3.5 THEN 'Honor'
        WHEN gpa >= 2.0 THEN 'Pass'
        ELSE 'At Risk'
    END AS classification,
    -- Nested CASE
    CASE
        WHEN gpa >= 3.0 THEN
            CASE WHEN dept_id = 1 THEN 'Eligible for CS scholarship'
                 WHEN dept_id = 2 THEN 'Eligible for Math scholarship'
                 ELSE 'Eligible for general scholarship'
            END
        ELSE 'Not eligible for scholarship'
    END AS scholarship_status
FROM student;
```

💻 Practice: Transaction as Error Handling

```sql
-- SQLite transactions provide atomicity
BEGIN TRANSACTION;

-- Check before insert (manual validation)
SELECT CASE
    WHEN EXISTS (SELECT 1 FROM student WHERE email = 'new@ytu.edu.tr')
    THEN 'Email already exists - would skip insert'
    ELSE 'Email available - would proceed with insert'
END AS validation_result;

COMMIT;

-- SAVEPOINT for partial rollback
BEGIN TRANSACTION;
SAVEPOINT sp1;
INSERT INTO student (first_name, last_name, dept_id, gpa)
VALUES ('Save', 'Point', 1, 3.0);
SAVEPOINT sp2;
INSERT INTO student (first_name, last_name, dept_id, gpa)
VALUES ('Save2', 'Point2', 1, 3.5);
ROLLBACK TO sp2;  -- Undo only the second insert
COMMIT;  -- First insert is kept

-- Verify
SELECT * FROM student WHERE first_name LIKE 'Save%';
DELETE FROM student WHERE first_name LIKE 'Save%';
```

Summary: MySQL Programming vs SQLite

MySQL Feature	SQLite Equivalent
`DECLARE var`	CTE or subquery
`SET @var = value`	CTE with computed column
`IF/ELSEIF/ELSE`	CASE expression in SELECT
`WHILE/REPEAT`	Recursive CTE
`DECLARE HANDLER`	INSERT OR IGNORE/REPLACE
`BEGIN...END`	BEGIN TRANSACTION...COMMIT
`DELIMITER //`	Not needed (no procedures)

Error Handling: MySQL vs SQL Server vs SQLite

Approach	MySQL	SQL Server	SQLite
Syntax	`DECLARE HANDLER FOR`	`BEGIN TRY...BEGIN CATCH`	Constraints + triggers
Scope	Event-specific	Block-level	Not supported natively
Error info	Limited	`ERROR_NUMBER/MESSAGE`	Via constraints
Transaction control	Implicit	Explicit in CATCH	`BEGIN/ROLLBACK`
Rollback on error	Optional (CONTINUE)	Automatic in CATCH	Manual transaction
Best for	Specific error handling	Comprehensive error management	Constraint validation

📝 Practice Exercises (Homework)

1. **CTE as variables:** Write a query that calculates the average GPA, then shows all students above average with their difference from average.
2. **CASE logic:** Create a grading report with letter grades, pass/fail status, and scholarship eligibility using nested CASE.
3. **Theory (exam prep):**
   - Explain the difference between DECLARE and @variable in MySQL
   - Write a MySQL procedure skeleton with IF/ELSE and WHILE
   - Explain why DELIMITER is needed before CREATE PROCEDURE

🔮 Next Week Preview

Week 11: Functions & Stored Procedures

- CREATE FUNCTION vs CREATE PROCEDURE
- IN, OUT, INOUT parameters
- DETERMINISTIC vs NOT DETERMINISTIC

Preparation:

- 📖 Read: MySQL documentation on stored routines
- 🤔 Think: What's the difference between a function and a procedure?

Thank You!

Questions?